

Anforderungen an das Projekt – Informatik 12 (gA)

LehrplanPLUS Inf 12 (grundlegendes Anforderungsniveau) – LB 6 (Praktische Softwareentwicklung, Projekt)
unter Anwendung der Inhalte aus LB 1–5

1. Rahmen und Lehrplanbezug

Das Projekt setzt Lernbereich 6 um und integriert verpflichtend Inhalte aus den Lernbereichen 1–5 (Rekursion, Listen, Binärbäume, Nebenläufigkeit, Informationssicherheit). Das Thema muss vor Beginn der Modellierung mit der Lehrkraft abgestimmt werden.

Rahmenbedingung	Vorgabe
Bearbeitung	Einzel- oder Partnerarbeit, max. 2 SuS
Bearbeitungszeit	ca. 26 Unterrichtsstunden + häusliche Fertigstellung
Programmiersprache	objektorientierte Sprache frei wählbar (z. B. Java, Python, C#)
Vorgehensmodell	Wasserfall oder agile Methode (Wahl pro Team, vorab dokumentiert)
Versionsverwaltung	verpflichtend (z. B. Git mit GitLab/GitHub; Plattform nach Absprache mit Lehrkraft), kein Accountzwang
Architektur	MVC erkennbar umgesetzt
Modellierung	UMLet, draw.io, PlantUML oder händisch
Abgabe	Quellcode (Repo-Link + ZIP) + Dokumentation (PDF) + Modellierung (PDF) über Lernplattform/ByCS

2. Themenvorschläge

Bereich	Beispielprojekte
Verwaltung	Inventar-, Bibliotheks-, Mitgliederverwaltung mit Suche und Sortierung
Spiele (GUI-basiert)	Vier gewinnt, Sudoku-Helfer, Memory mit Highscore, kleines Strategie- oder Reaktionsspiel
Smarthome-Simulation	Lichtsteuerung, Heizungsplaner, virtueller Sensor-Hub
Datenstruktur-Anwendungen	Musikplayer-Playlist (verkettete Liste), Wörterbuch (Binärbaum), Restaurant-Bestellsystem (Queue)
Produktivität	Karteikartensystem mit Lernalgorithmus, To-Do-Manager mit Prioritätsbaum

3. Verbindliche Anforderungen aus dem Lehrplan

3.1 Vorgehensmodell und Projektmanagement (LB 6)

#	Anforderung	Mindestens
V1	Vorgehensmodell festlegen (Wasserfall oder agile Methode, z. B. Scrum-light) und schriftlich begründen	1×
V2	Wasserfall: Pflichtenheft mit Zielsetzung, funktionalen und nicht-funktionalen Anforderungen. Agil: User Stories + Tasks + Project Board	≥ 6 User Stories bzw. vollständiges Pflichtenheft
V3	Meilensteinplan (Wasserfall) bzw. Sprint-Plan mit Retrospektive (agil)	≥ 3 Meilensteine / Sprints
V4	Reflexion zum gewählten Modell: Was lief gut, wo waren Grenzen?	½ Seite

3.2 Architektur und Implementierung (LB 6)

#	Anforderung	Mindestens
I1	Anwendung mit grafischer Benutzeroberfläche (GUI)	≥ 4 Bedienelemente
I2	Bibliotheksklassen sinnvoll einsetzen, Verweis auf die offizielle Dokumentation	≥ 2 Klassen
I3	MVC-Architektur erkennbar: getrennte Klassen für Model, View, Controller; Verantwortlichkeiten dokumentiert	umgesetzt
I4	Persistente Datenspeicherung (Datei oder DBMS): Lesen und Schreiben	umgesetzt

I5	Versionsverwaltung: Repository mit aussagekräftigen Commit-Messages, regelmäßige Commits (nicht nur am Ende)	≥ 10 Commits, ≥ 4 verschiedene Arbeitstage
-----------	--	--

3.3 Test und Refaktorisierung (LB 6)

#	Anforderung	Mindestens
T1	Testkonzept dokumentiert (was wird wie getestet)	1 Seite
T2	Manuelle Testfälle in Testprotokoll (Schritte, erwartetes/tatsächliches Ergebnis)	≥ 5 Testfälle
T3	Automatisierte Komponententests (z. B. JUnit, pytest, NUnit)	≥ 2 Tests
T4	Refaktorisierungsschritte dokumentiert mit Vorher/Nachher-Code-Ausschnitt und Begründung (Lesbarkeit, Redundanzvermeidung, Erweiterbarkeit)	≥ 2 Schritte

3.4 Anwendung von Inhalten aus LB 1–5

#	Anforderung	Mindestens
L1	Eigene Implementierung einer Datenstruktur aus LB 2 oder LB 3 (einfach verkettete Liste, Stack, Queue oder geordneter Binärbaum) sinnvoll im Projekt eingesetzt – Bibliotheks-Listen ersetzen diese Pflicht nicht	1 Datenstruktur
L2	Rekursiver Algorithmus (LB 1) im Projekt verwendet (z. B. rekursive Listen-/Baumoperation, Tiefensuche, selbstähnliche Grafik)	1 Algorithmus
L3	Informationssicherheit (LB 5): Reflexion mit Anwendung von ≥ 2 Schutzziele (Vertraulichkeit, Integrität, Verfügbarkeit, Authentizität) auf das eigene Projekt; Benennung mind. einer Gefährdung und einer Maßnahme	½–1 Seite
L4	Nebenläufigkeit (LB 4) – optional: mind. ein eigener Thread mit dokumentierter Synchronisation oder Vermeidung kritischer Abschnitte (gibt Bonuspunkte)	optional

Hinweis: L1 und L2 können kombiniert werden (z. B. rekursive Einfüge-Methode in der eigenen verketteten Liste deckt beides ab).

4. Projektorganisation

#	Anforderung	Prüfbares Kriterium
O1	Aufgabenteilung	Bei Partnerarbeit dokumentiert, wer welchen Teil verantwortet
O2	Projektplan	Aufgabenpakete, Schnittstellen, Verantwortlich, Meilensteine
O3	Versionsverwaltung	Repo-Link in Doku; kontinuierliche Commit-Historie
O4	Iterative Entwicklung	Klassendiagramm steht vor Implementierung; Änderungen im Entwicklungsverlauf sichtbar
O5	Test im Verlauf	Tests werden parallel zur Implementierung entwickelt (nicht erst am Ende)

5. Einsatz von Künstlicher Intelligenz

Der Einsatz von KI ist ausdrücklich erlaubt und erwünscht, jedoch ausschließlich über die Werkzeuge, die der Freistaat Bayern zentral und datenschutzkonform bereitstellt:

KI-Werkzeug	Zugang	Status
KI-Funktionalitäten der ByCS-Lernplattform (ByCS-Standard-Tool, FWU)	über ByCS, Freischaltung durch Lehrkraft	erlaubt
Schul-Chatbot „telli“ (FWU)	über VIDIS in der ByCS	erlaubt
Kommerzielle Anbieter (z. B. ChatGPT, Claude.ai, Gemini, Microsoft Copilot, GitHub Copilot, Perplexity)	extern	nicht zulässig

Pflichten beim KI-Einsatz:

#	Pflicht
KI-1	Alle für das Projekt relevanten KI-Prompts werden im Anhang vollständig dokumentiert: Tool, Datum, vollständiger Prompt-Wortlaut, übernommene/abgelehnte Ausgabe, eigene Bearbeitung der KI-Antwort.
KI-2	KI-generierte Code-Abschnitte sind im Code als solche zu kennzeichnen (Kommentar mit Quelle, Datum, Bearbeitungsstand).
KI-3	Die Eigenleistung muss nachvollziehbar bleiben – insbesondere Architekturentscheidungen, Implementierung der eigenen Datenstruktur und Refaktorisierungen sind eigene Arbeit.
KI-4	Bei Verstoß gegen KI-1 bis KI-3 gelten die schulüblichen Regeln zur Täuschung; die Leistung kann mit „ungenügend“ bewertet werden.

6. Dokumentation (PDF, max. 6 Seiten zzgl. Anhänge)

Abschnitt	Inhalt
1. Deckblatt	Projekttitel, Namen, Klasse, Vorgehensmodell, Datum, Repo-Link
2. Aufgabenstellung & Szenario	Problembeschreibung, funktionale und nicht-funktionale Anforderungen
3. Vorgehensmodell	Begründung der Wahl; Pflichtenheft (Wasserfall) oder User Stories + Sprintplan (agil)
4. Architektur & Modellierung	Klassendiagramm mit MVC-Schichten; ggf. Sequenzdiagramm
5. Implementierung	Bibliotheksklassen mit Quellenangabe; Hervorhebung der eigenen Datenstruktur und des rekursiven Algorithmus
6. Entwicklungsverlauf	Nachvollziehbare Darstellung des Wegs vom ersten Konzept zum fertigen Produkt: Konzept → Modellierung → Implementierung → Test → Refaktorisierung → Endprodukt. Aufgetretene Probleme und Lösungen; wichtige Designentscheidungen. Belege durch Screenshots oder Zwischenversionen sind erwünscht.
7. Test	Testkonzept, manuelles Testprotokoll, Komponententests (Ausschnitt)
8. Refaktorisierung	≥ 2 dokumentierte Schritte mit Vorher/Nachher-Code und Begründung
9. Informationssicherheit	Reflexion zu Schutzzielen, Gefährdungen, Maßnahmen für das eigene Projekt
10. Arbeitsteilung	Wer hat was gemacht?
11. Reflexion	Vorgehensmodell, Teamarbeit, Lernerfolg
Anhang A	KI-Protokoll gemäß KI-1
Anhang B	Git-Log oder Auszug daraus

7. Bewertung (Vorschlag, 100 Punkte)

Bereich	Punkte
Vorgehensmodell und Projektmanagement (V1–V4)	8
Architektur und Implementierung (I1–I5, inkl. MVC, GUI, Persistenz, Versionsverwaltung)	22
Test und Refaktorisierung (T1–T4)	12
Anwendung aus LB 1–5 (L1 eigene Datenstruktur, L2 Rekursion, L3 Informationssicherheit)	15
Funktionalität (lauffähig, sinnvolle Bedienoberfläche, Beispielablauf)	12
Code-Qualität (Lesbarkeit, Namenskonventionen, Kommentare, MVC-Sauberkeit)	8
Projektorganisation (O1–O5, Qualität der Versionsverwaltung)	7
Dokumentation inkl. Entwicklungsverlauf + KI-Protokoll	13
Bonus: Nebenläufigkeit (L4)	+3
Summe	100 (+3)

Notenschlüssel (Vorschlag): 1 ab 85 P · 2 ab 70 P · 3 ab 55 P · 4 ab 40 P · 5 ab 20 P · 6 unter 20 P.

8. Abgabe-Checkliste (Selbstkontrolle)

- ☐ Team aus max. 2 SuS, Thema vorab abgestimmt
- ☐ Vorgehensmodell festgelegt; Pflichtenheft oder User Stories + Sprintplan vorhanden
- ☐ Klassendiagramm mit erkennbaren MVC-Schichten
- ☐ Anwendung mit GUI (≥ 4 Bedienelemente) läuft fehlerfrei
- ☐ ≥ 2 Bibliotheksklassen sinnvoll eingesetzt, Quellen dokumentiert
- ☐ Persistente Datenspeicherung (Datei oder DB) funktioniert
- ☐ Repository mit ≥ 10 aussagekräftigen Commits; Repo-Link in der Doku
- ☐ Eigene Datenstruktur aus LB 2 oder LB 3 implementiert und im Projekt eingesetzt
- ☐ Rekursiver Algorithmus enthalten und im Code markiert
- ☐ Reflexion zur Informationssicherheit (≥ 2 Schutzziele angewandt)
- ☐ ≥ 5 manuelle Testfälle + ≥ 2 automatisierte Komponententests
- ☐ ≥ 2 Refaktorisierungsschritte mit Vorher/Nachher dokumentiert
- ☐ KI-Einsatz ausschließlich über ByCS / telli; vollständiges KI-Protokoll im Anhang
- ☐ Entwicklungsverlauf dokumentiert (Konzept $\rightarrow$... $\rightarrow$ Endprodukt mit Designentscheidungen)
- ☐ Dokumentation vollständig (max. 6 S. + Anhänge)
- ☐ Quellcode (Repo-Link + ZIP), Modellierung (PDF), Doku (PDF) im richtigen Abgabeordner